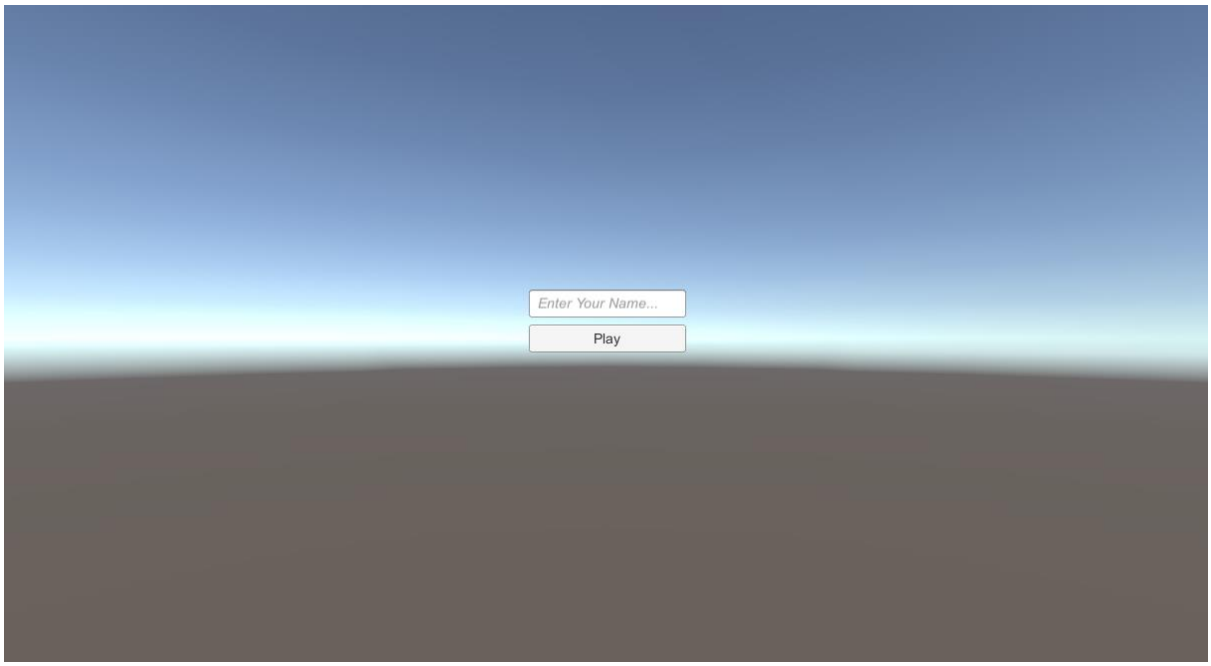
Launcher

To start the test, I followed the tutorial on the photon. With it, I was able to connect to their servers and enter/ create rooms.

I started by creating a launcher scene where you can put your name and connect to the server.



I then created a launcher script where I create the logic to connect to the servers.

```
/// <summary>
/// Start the connection process.
/// - If already connected, we attempt joining a random room
/// - if not yet connected, Connect this application instance to Photon Cloud Network
/// </summary>
0 references
public void Connect()
{
    progressLabel.SetActive(true);
    controlPanel.SetActive(false);

    // we check if we are connected or not, we join if we are , else we initiate the connection to the server.
    if (PhotonNetwork.IsConnected)
    {
        // #Critical we need at this point to attempt joining a Random Room. If it fails, we'll get notified in OnJoinRandomFailed() and we'll create one.
        PhotonNetwork.JoinRandomRoom();
    }
    else
    {

        // #Critical, we must first and foremost connect to Photon Online Server.
        // keep track of the will to join a room, because when we come back from the game we will get a callback that we are connected, so we need to know what to do then
        isConnecting = PhotonNetwork.ConnectUsingSettings();
        PhotonNetwork.GameVersion = gameVersion;
    }


}
```

Every time the player tries to connect, it will detect if a game is already running online.

```
3 references
public override void OnJoinedRoom()
{
    // #Critical: We only load if we are the first player, else we rely on `PhotonNetwork.AutomaticallySyncScene` to sync our instance scene.
    if (PhotonNetwork.CurrentRoom.PlayerCount == 1)
    {
        Debug.Log("We load the 'Playground' ");

        // #Critical
        // Load the Room Level.
        PhotonNetwork.LoadLevel("Playground");
    }
}
```

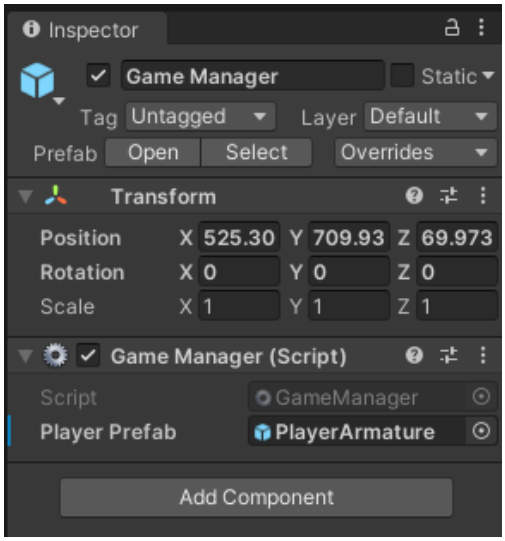
if no game is running, it will create a room a new room for the game.

```
3 references
public override void OnJoinRandomFailed(short returnCode, string message)
{
    Debug.Log("OnJoinRandomFailed() was called by PUN. No random room available, so we create one.\nCalling: PhotonNetwork.CreateRoom");

    // #Critical: we failed to join a random room, maybe none exists or they are all full. No worries, we create a new room.
    PhotonNetwork.CreateRoom(null, new RoomOptions { MaxPlayers = maxPlayersPerRoom });
}
```

Game Manager

Then I used the Third Person Scene of StartAssets to create the playable environment.

There I create a Game Manager that takes care of connecting and creating all the players' characters.



There are some methods in this class but the one I want to point out is the Start method, where the manager creates the player character from a prefab located on the resources folder.

```
Unity Message | 0 references
void Start()
{

    if (playerPrefab == null)
    {
        Debug.LogError("<Color=Red><a>Missing</a></Color> playerPrefab Reference. Please set it up in GameObject 'Game Manager'", this);
    }
    else
    {
        Debug.LogFormat("We are Instantiating LocalPlayer from {0}", Application.loadedLevelName);
        // we're in a room. spawn a character for the local player. it gets synced by using PhotonNetwork.Instantiate

        PhotonNetwork.Instantiate(this.playerPrefab.name, new Vector3(0f, 0f, 0f), Quaternion.identity, 0);

    }
}
```

There are also methods on Player joined/left the room, but I didn't use them, just in case it's needed and for log purposes.

```
3 references
public override void OnPlayerEnteredRoom(Player other)
{
    Debug.LogFormat("OnPlayerEnteredRoom() {0}", other.NickName); // not seen if you're the player connecting

if (PhotonNetwork.IsMasterClient)
    {
        Debug.LogFormat("OnPlayerEnteredRoom IsMasterClient {0}", PhotonNetwork.IsMasterClient); // called before OnPlayerLeftRoom

    }
}


3 references
public override void OnPlayerLeftRoom(Player other)
{
    Debug.LogFormat("OnPlayerLeftRoom() {0}", other.NickName); // seen when other disconnects


    if (PhotonNetwork.IsMasterClient)
    {
        Debug.LogFormat("OnPlayerLeftRoom IsMasterClient {0}", PhotonNetwork.IsMasterClient); // called before OnPlayerLeftRoom


    }
}
```

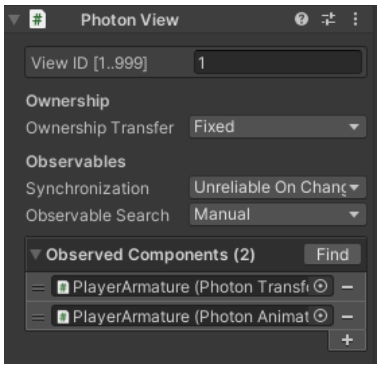
Player Character

This is the part where I spend more time since here everything is mostly custom.

For the player, I started by adding some photon components.

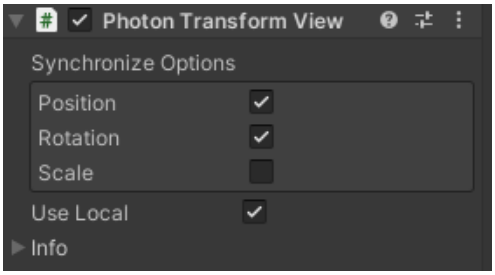
Photon View

Connects to the rest of the photon components and communicates with the server.



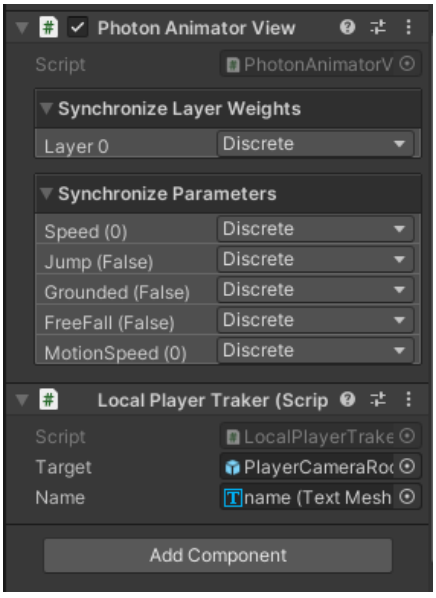
Photon Transform View

Tracks the transform of the object (position, rotation, …)



Photon animator view

Tracks all the animation variables and synchronises them with the other players (it made it really easy, you just add them to the object, and it detects and animator component automatically)



Basically, these three components made it so all the characters are synchronised.

The only problem I had was with the animation because the character movement code was overwriting it.

I just had to add a check on the character update to see if the character was owned by the local player and then progress as normal.

```
Unity Message | 0 references
private void Update()
{
    if (GetComponent<PhotonView>().IsMine == false && PhotonNetwork.IsConnected == true)
    {
        return;
    }
```

Camera

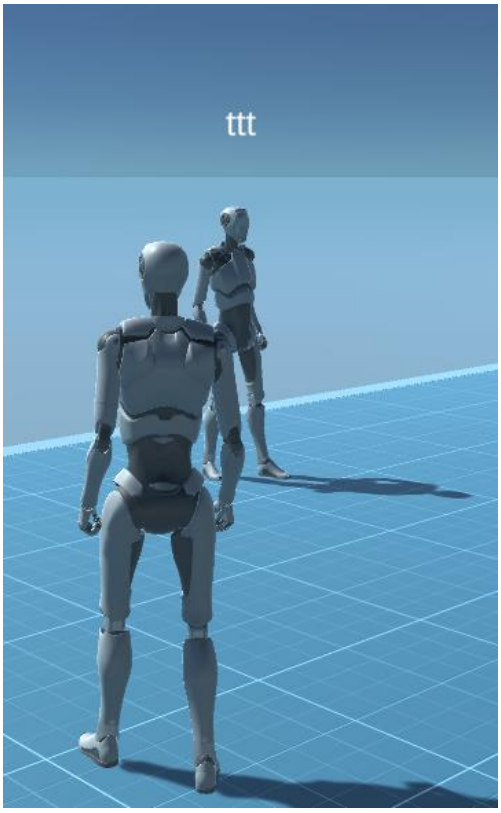
To make the camera work, I just add to create a new script for the player character where it detects if I'm the owner of the character and then assig the tracker to the CinemachineVirtualCamera.

```
if (localPhoton.IsMine)
{
    GameObject.Find("PlayerFollowCamera").GetComponent<CinemachineVirtualCamera>().Follow = Target.transform;
```

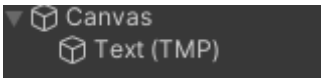
This was one of the parts where I had more problems mostly because I didn't know how to detect if I'm the owner of the character.

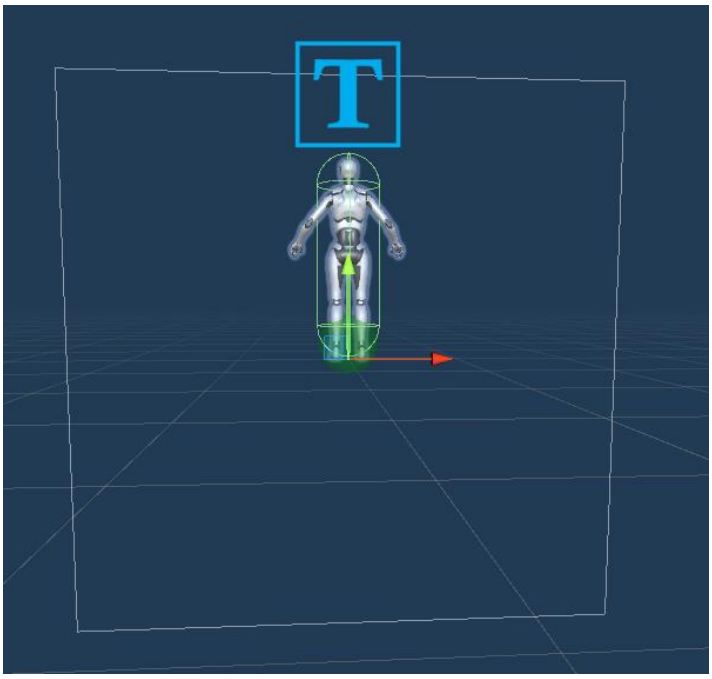
Nick Name

To identify the characters, put the owners' names on top of them.



To achieve this, I started by adding a canvas, in real word space, to the players, with a TPM UI text.





Then on the Local player tracker script, I modified the text with the nicknames stored on the photo view component.

```
localPhoton = GetComponent<PhotonView>();

// #Important
// used in GameManager.cs: we keep track of the localPlayer instance to prevent instantiation when levels are synchronized
if (localPhoton.IsMine)
{
    GameObject.Find("PlayerFollowCamera").GetComponent<CinemachineVirtualCamera>().Follow = Target.transform;

    Name.gameObject.transform.parent.gameObject.SetActive(false);
}
else
{
    Name.text = localPhoton.Owner.NickName;
}
```

And made it so the canvas always faces the camera.

```
Unity Script (1 asset reference) | 0 references
public class PlayerUIRoate : MonoBehaviour
{


    // Update is called once per frame
    Unity Message | 0 references
    void Update()
    {
        GetComponent<RectTransform>().LookAt(Camera.main.transform.position);
    }
}
```

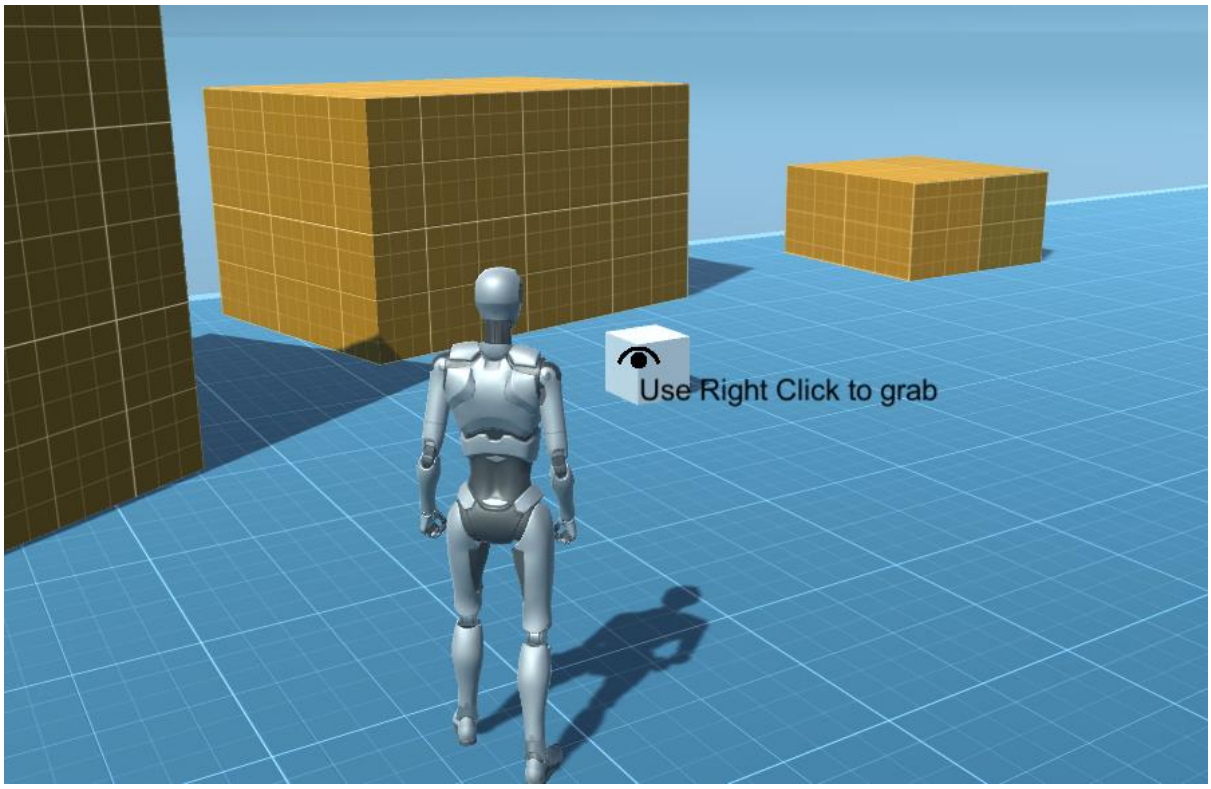
Extras

Box Pick up

This was the part where I had more difficulties.

Mostly because I didn't know how to send custom information at first.

I started by creating a prefab of a box that can be targetable. To identify that I used a tag, and used a ray cast from the camera.



Then I just use the right click of the mouse to pick it up.

For this, to work I used 2 scripts "GrabItems" and "CubesLogic".

Grab Items takes care of targeting the cubes and picking them up.

I use states to optimise as much as possible.

```
9 references
enum GrabedState
{
    Attracting,
    Connected,
    Null
}
```

Null state

The Null state just uses the ray cast and detects the mouse click.

```
case GrabedState.Null:

    //Detect Raycast collision
    if (Physics.Raycast(ray, out hit) && hit.transform.tag == "throwable")
    {

        //Detect distance between the player and cube
        if (Vector3.Distance(transform.position, hit.transform.position) <= GrabbingDistance)
        {
            UI.SetActive(true);

            //Detect mouse click
            if (photonview.IsMine && Mouse.current.rightButton.wasPressedThisFrame)
            {
                // Change Grab Item State
                grabedState = GrabedState.Attracting;

                GrabbedItem = hit.transform.gameObject;
                GrabbedItem.GetComponent<PhotonView>().TransferOwnership(photonview.Controller);
                GrabbedItem.GetComponent<Rigidbody>().useGravity = false;

                //Change Cube State
                GrabbedItem.GetComponent<CubesLogic>().cubestate = Cubestate.Grabed;
            }
        }
    }
    else
        UI.SetActive(false);
    break;
```

Attracting State

I wanted to create the effect of the cube coming to you instead of just teleporting it, so I created the attracting state where I used Vector3 lerp to slowly drag it to you.

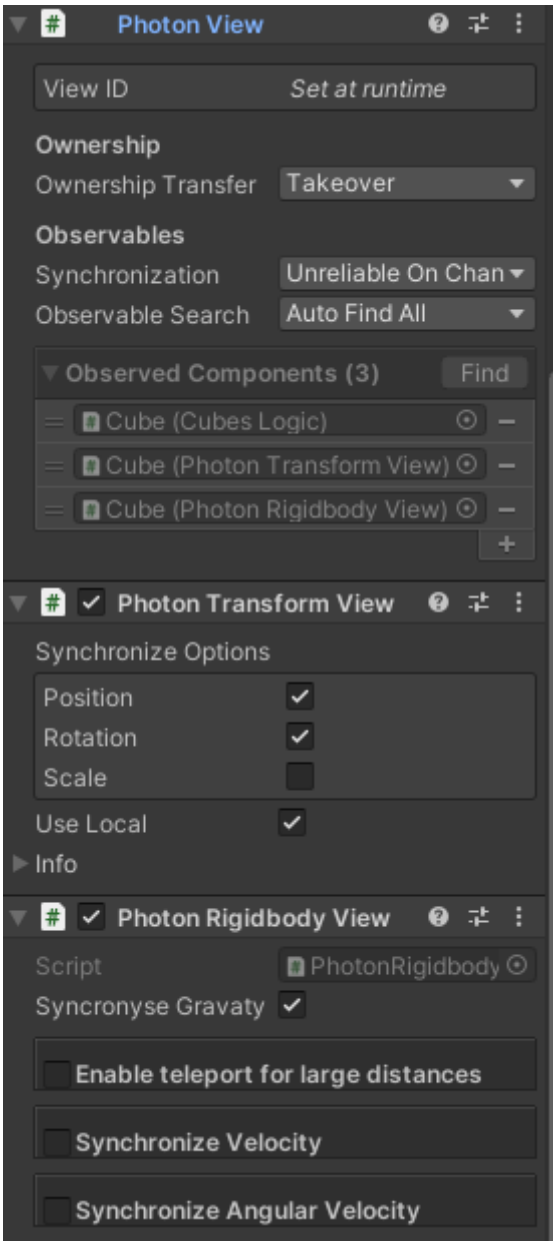
```
case GrabedState.Attracting:

    //Slowly move item
    GrabbedItem.transform.position = Vector3.Lerp(GrabbedItem.transform.position, transform.position, AttractingSpeed*Time.deltaTime);

    //Snap it to place
    if (Vector3.Distance(transform.position, GrabbedItem.transform.position) <= GrabingAdjustDistance)
    {
        GrabbedItem.transform.parent = transform;

        //Update GabItem State
        grabedState = GrabedState.Connected;
    }

    break;
```

Connect State

Does basically the same thing as the previous one but add the possibility to right-click to release.

```
case GrabedState.Connected:

    GrabbedItem.transform.position = Vector3.Lerp(GrabbedItem.transform.position, transform.position, AttractingSpeed * Time.deltaTime);

    if (Vector3.Distance(transform.position, GrabbedItem.transform.position) <= GrabingAdjustDistance)
    {
        GrabbedItem.transform.localPosition = Vector3.zero;
    }



    if (photonview.IsMine && Mouse.current.rightButton.wasPressedThisFrame)
    {
        GrabbedItem.transform.parent = null;

        GrabbedItem.GetComponent<Rigidbody>().useGravity = true;

        grabedState = GrabedState.Null;

        GrabbedItem.GetComponent<CubesLogic>().cubestate = Cubestate.grounded;

        GrabbedItem = null;
    }
```

Now the difficult part syncing, at first, I thought it would be easy, I was thinking of using the photon components to track the position and update it.



But I figure out quickly that it wasn't enough since I was applying logic on the cubes that weren't being tracked, like the tags, or the gravity on the rigid body.

After learning how sending and receiving info worked, I tried to use it on the GrabItem script, by updating the States (good organisation of my past self ;D) and updating the game object target.

```
public void OnPhotonSerializeView(PhotonStream stream, PhotonMessageInfo info)
{
    if (stream.IsWriting)
    {

        // We own this player: send the others our data
        stream.SendNext(GrabbedItem);
        stream.SendNext(grabedState);
    }
    else
    {
        // Network player, receive data
        this.GrabbedItem = (GameObject)stream.ReceiveNext();
        this.grabedState = (GrabedState)stream.ReceiveNext();
    }
}
```

But unfortunately, you can't pass gameOjects.

So, I went to the cubes instead, first to solve the gravity problem, I changed the photon script by adding a gravity track.

```
if(SyncronyseGravaty)
stream.SendNext(this.m_Body.useGravity);
```

Then I used the Cube Logic script to update the tags and basically make them untreatable to other players while grabbed.

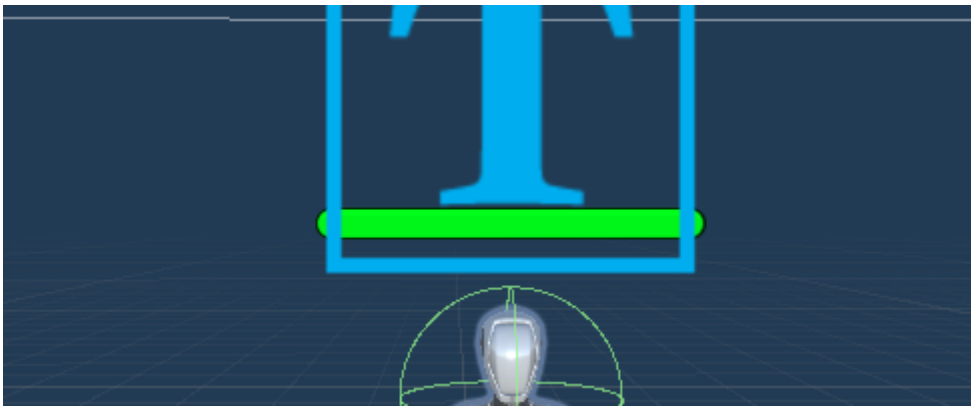
```
switch (cubestate)
{
    case Cubestate.Grabed:

        tag = "Untagged";

        break;

    case Cubestate.Lanched:

        if (Rigidbody.velocity == Vector3.zero)
        {
            cubestate = Cubestate.grounded;

        }
        break;

    case Cubestate.grounded:
        tag = "throwable";

        if (transform.parent)
            transform.parent = null;

        break;
    default:
        break;
```

```
3 references
public void OnPhotonSerializeView(PhotonStream stream, PhotonMessageInfo info)
{
    if (stream.IsWriting)
    {
        // We own this player: send the others our data
        stream.SendNext(cubestate);
        stream.SendNext(ParentName);


    }
    else
    {
        // Network player, receive data
        this.cubestate = (Cubestate)stream.ReceiveNext();

        this.ParentName = (string)stream.ReceiveNext();

        if (cubestate == Cubestate.Grabed && transform.parent)
        {

            GameObject[] temp = GameObject.FindGameObjectsWithTag("Player");

            //Detect with player owns the cube being garbed
            foreach (var item in temp)
            {
                if(item.GetComponent<PhotonView>().Owner == photonView.Owner)
                {
                    transform.parent = item.transform.GetChild(0).GetChild(0);
                }
            }

        }
```

Since I was having some problems with the cube not being completely fixed on the eyes of other players, I tried to snap them to the pivot by making it parent but there are some issues still.

Health

For this, I just used a slider as a health indicator on the player.



And for code, I used the photon method to send information.

```
3 references
public void OnPhotonSerializeView(PhotonStream stream, PhotonMessageInfo info)
{

    if (stream.IsWriting)
    {
        // We own this player: send the others our data

        stream.SendNext(CurrentHealth);

    }
    else
    {
        // Network player, receive data
        this.CurrentHealth = (int)stream.ReceiveNext();

    }
}
```

You can throw the cubes to other players to damage them